A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science & Engineering

Under the Course of-

Course Title: Thesis/Project Work
Course Code: CSE 4000(A)

Thesis Supervisor-
Name: Mst. Sahela Rahman
Designation: Lecturer

Prepared by-
Name: Md. Riaz
ID/Registration No: 0322310105101024
Semester, Year: 4th Year - 7th Semester
Session: Spring - 2023

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Date of submission: _____

A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science & Engineering

Signature of Supervisor

Signature of Student

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Date of submission: _____

CERTIFICATION OF ORIGINALITY

I hereby declare that this thesis titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics" is my own original work, carried out under the supervision of Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology. To the best of my knowledge, this thesis contains no material previously published or written by another person, except where due reference is made in the text. This work has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

_____

Signature of Student

Md. Riaz

ID: 0322310105101024

CERTIFICATION OF APPROVAL

This is to certify that the research paper titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

That it has been read, evaluated and accepted for fulfilment of the Academic Degree of Bachelor in Computer Science and Engineering (B.S.C. in CSE) at Pundra University of Science & Technology.

The acceptance decision is made based upon an independent review process that provides critically constructive feedback within a short turnaround time. This paper represents the author's independent work, critical analysis, and capability in undertaking literature research as per the academic policies and ethical standards of the university.

I certify that the candidate has completed the required research activities for the program and recommend that this piece of work as of acceptable standard for submission and for inclusion in the academic record of the University.

Mst. Sahela Rahman
Lecturer
Department of Computer Science and Engineering
Pundra University of Science & Technology

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology, for her continuous guidance, patience, and constructive feedback throughout the design, implementation, and evaluation of this research. Her insistence on precise, defensible claims shaped this thesis at every stage, from the formal safety proof to the honesty of its discussion of limitations.

I am also grateful to the faculty members of the Department of Computer Science & Engineering for the coursework and feedback that prepared me to undertake this research, and to my family for their patience and encouragement during the long implementation and evaluation cycles this thesis required.

Finally, I acknowledge the authors whose published research on natural language interfaces, text-to-SQL translation, and dashboard generation provided the foundation against which AEGIS's contributions are measured.

ABSTRACT

Relational databases hold data organizations need for decisions, but non-technical users often depend on developers to translate business questions into SQL. Natural-language interfaces try to close this gap, but many current systems still allow a language model to author executable SQL directly, making safety dependent on model behavior rather than system structure. This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), an architecture that restricts the model to intent extraction while deterministic software handles semantic mapping, permission enforcement, SQL compilation, validation, visualization selection, and widget persistence. AEGIS uses a closed semantic layer of approved metrics, dimensions, analytical patterns, and join paths, so the model never outputs SQL text. Instead, it produces a typed intent object that can be checked before any query is compiled. A prototype evaluation over a 107-query mixed natural-language benchmark on a seeded nopCommerce-style database shows that AEGIS produced no unsafe SQL statements and achieved 100 successful true database executions out of 107, while a direct LLM-to-SQL baseline achieved 27 successful executions out of 107 and produced one genuine unsafe write statement. These results support the thesis's central claim that SQL safety can be made a structural property of the architecture. The evaluation also shows a remaining limitation: safe and executable SQL is not always semantically correct, so correctness and scope-handling require a separate annotated benchmark in future work.

Table of Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1.1 Background

Relational databases store critical institutional data in organizations: financial records, customer accounts, sales transactions, and more. But accessing this data is uneven. Technical staff can write SQL queries to get any answer they need, while non-technical users have to wait for someone else to build them a report. This waiting is expensive. Analysis of enterprise reporting workflows shows that business users frequently wait days for new reports, and a recurring theme in institutional reporting is that many questions are variations of things already asked before: the same report with a different date range, or the same chart for a different department. These are not one-off questions; they are recurring reporting needs that should be served by saved, refreshable widgets rather than regenerated from scratch every time.

This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that lets users describe their reporting needs in plain English and produces dynamic dashboard widgets that can be saved, refreshed, and reused as part of their daily workflow, without anyone writing SQL.

Natural language interfaces to databases (NLIDBs) try to solve this problem. The idea is simple: a user should be able to ask "which categories have the highest refund rates this month?" and get a correct, visual answer without writing SQL. Researchers have made good progress here through benchmarks such as Spider [1] and BIRD [2]. But there is still a gap between benchmark results and production-ready deployment.

1.2 Problem Statement

Three problems make up the gap between benchmark text-to-SQL accuracy and safe, production-ready natural-language reporting.

- **Safety:** Models may generate SQL directly, which can expose private data or create wrong joins.

- **Vocabulary mismatch:** Users ask with business terms such as "refund rate", while systems often expect column names.
- **No reusable widgets:** Most systems answer once and discard the result instead of saving refreshable dashboard widgets.

These problems are not primarily about building a smarter model; they are about designing the system properly around the model. AEGIS addresses this by splitting the work into stages. The LLM's only job is to understand what the user is asking and output a structured description of the request. Everything after that, matching to the right business terms, building the SQL, selecting the chart, and saving the widget, is done by fixed rules and pre-approved templates.

1.3 Research Novelty and Motivation

Existing text-to-SQL research asks: how accurately can a model generate SQL from natural language? This thesis asks a different question: how can large language models be used for language understanding while being structurally prevented from generating executable SQL at all? The two pipelines differ structurally.

Classical NL-to-SQL: natural-language request, then model-authored SQL, then query result.

AEGIS: natural-language request, then intent extraction, semantic constraint, deterministic compilation, and a safe analytical artifact.

The contribution of this thesis is therefore not improved SQL generation accuracy; it is constrained analytical artifact generation, a design approach that removes SQL generation from the LLM's role entirely and provides safety and semantic fidelity guarantees that no generative model can match unconditionally.

AEGIS does not propose a new LLM. The model is interchangeable: Groq-hosted Llama 3.1 8B, OpenRouter, a local Ollama instance, or any endpoint compatible with the /v1/chat/completions interface. Because the LLM's only contract with the rest of the system is to produce a typed JSON intent object, model upgrades improve quality automatically without changing the compiler or safety infrastructure. This is model independence by design, not an implementation accident.

1.4 Objectives and Contributions

This thesis makes the following contributions:

1. A design-time review of representative e-commerce and business-intelligence reporting requests, resulting in eleven common reporting patterns later used to structure the custom 107-request benchmark.
2. A system design in which all possible queries are limited to pre-approved templates and a defined semantic layer, which prevents SQL injection and unauthorized data access by construction.
3. The AEGIS system itself, including the semantic layer design, a vocabulary injection prompting strategy, a safe SQL builder with two-layer defence, a rule-based chart selector, and a widget storage system with scheduled refresh.
4. A vocabulary injection method that places approved metric and dimension names directly into the LLM prompt, reducing dependence on a manually written synonym list.
5. A verified benchmark over 107 mixed natural-language requests, separating SQL safety, true database execution validity, and semantic correctness rather than treating them as one accuracy number.
6. A planned cross-schema generalizability evaluation that will test whether the same compiler and safety architecture can be reused on a second e-commerce schema by rebuilding only the semantic layer.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

The literature review focuses on the work most directly comparable to AEGIS. It is organized as short review blocks so each source can be checked quickly by contribution, limitation, and the specific gap it leaves for this thesis.

2.1 Natural Language Interfaces to Databases

NLIDB surveys [3, 4]

- **Contribution:** Compare major natural-language database interface designs, including keyword, pattern, parsing, and grammar-based systems.
- **Limitations:** Safety is treated mostly as a secondary concern; even SQL-injection discussion ends with generic filtering advice.
- **Gap for AEGIS:** No reviewed system makes SQL safety a structural design property.

NaLIR [5]

- **Contribution:** Uses an intermediate query tree and asks the user to choose between possible interpretations.
- **Limitations:** Correctness depends on user disambiguation, and the final output remains executable SQL.
- **Gap for AEGIS:** Ambiguity is handled after parsing, but unsafe or unauthorized queries are not prevented by design.

Veezoo [6]

- **Contribution:** Uses an editable Knowledge Graph that maps business language to database concepts.
- **Limitations:** The semantic layer improves usability, but the paper does not evaluate SQL safety or permission enforcement.
- **Gap for AEGIS:** A semantic layer exists in prior work, but not as a safety boundary.

2.2 Neural and LLM-Based Text-to-SQL

Spider and BIRD [1, 2]

- **Contribution:** Provide large text-to-SQL benchmarks for complex and cross-domain database questions.
- **Limitations:** They measure SQL correctness and execution accuracy, not whether a query should be allowed.
- **Gap for AEGIS:** Benchmark success does not equal database safety or business-policy compliance.

RAT-SQL [7]

- **Contribution:** Models schema relations directly so table and column linking can improve on complex schemas.
- **Limitations:** Errors still frequently come from wrong table or column selection.
- **Gap for AEGIS:** Better schema encoding reduces some mistakes but does not create an authorization boundary.

PICARD [8]

- **Contribution:** Constrains decoding so generated SQL is more likely to be syntactically valid.
- **Limitations:** The model still authors the SQL, and grammar validity does not prove the query is semantically safe.
- **Gap for AEGIS:** A valid SQL string can still express the wrong intent or access the wrong data.

G-SQL and TriSQL [9, 10]

- **Contribution:** Show two strong directions: rule-guided schema-aware translation and multi-stage LLM refinement.
- **Limitations:** G-SQL has limited qualitative evaluation, while TriSQL still depends on LLM-authored executable SQL.
- **Gap for AEGIS:** The closest prior work improves accuracy, but does not remove SQL generation from the LLM output space.

2.3 Natural Language for Visualization and Dashboards

nl4dv and DataTone [11, 12]

- **Contribution:** Map natural-language questions into visualization tasks and expose ambiguity instead of silently guessing.
- **Limitations:** They focus on visualization specification, not protected database execution or persistent dashboard widgets.
- **Gap for AEGIS:** Visualization systems solve chart selection but leave SQL safety and reuse outside the architecture.

DashBot [13]

- **Contribution:** Uses deep reinforcement learning and dashboard-design rules to compose multi-chart dashboards.
- **Limitations:** It has no natural-language-to-SQL stage and does not handle governed production database access.
- **Gap for AEGIS:** Dashboard composition is useful, but it does not solve safe language-to-data translation.

2.4 Applied Conversational Business Intelligence

Conversational BI assistants [14]

- **Contribution:** Demonstrate that LLM-based chat can be connected to dashboards and SQL tools.
- **Limitations:** Direct SQL execution through an LLM tool loop creates a large attack surface and is rarely evaluated adversarially.
- **Gap for AEGIS:** A production assistant needs a constrained execution layer, not only a conversational interface.

Explanatory dashboard analytics [15]

- **Contribution:** Shows that deterministic computation followed by LLM narration can improve reliability in business explanation tasks.
- **Limitations:** The work targets dashboard explanation, not safe SQL compilation or reusable widget creation.
- **Gap for AEGIS:** It supports AEGIS's choice to let deterministic code compute results while the LLM handles language.

2.5 Comparative Summary

Table 1: Comparative summary of the most closely related systems.

System	NL	Semantic	Safe SQL	Visual	Widget	Coverage	Evaluation
Spider/BIRD	Yes	-	-	-	-	-	Benchmark
RAT-SQL/PICARD	Yes	-	Partial	-	-	-	Benchmark
G-SQL/TriSQL	Yes	Partial	Partial	-	-	-	Benchmark
NaLIR	Yes	-	-	-	-	-	User study
Veezoo	Yes	Yes	-	-	-	-	User study
nl4dv/DataTone	Yes	-	-	Yes	-	-	User study
DashBot	-	-	-	Yes	Partial	-	Synthetic
Conversational BI	Yes	-	-	Yes	-	-	Demo
AEGIS	Yes	Yes	Yes	Yes	Yes	Yes	Production

2.6 Research Gap Analysis

- **Safety gap:** Most systems measure answer accuracy, but do not evaluate unsafe SQL behavior.
- **Semantic-layer gap:** Semantic layers appear in prior work, but mainly for usability and matching, not as execution boundaries.
- **Visualization gap:** Visualization systems produce charts, but usually operate outside governed SQL execution.
- **Persistence gap:** Prior tools often answer one question at a time, while recurring business reports need refreshable outputs.
- **Thesis position:** AEGIS addresses these gaps through a bounded vocabulary, deterministic query compiler, safe visualization selector, and reusable widget model.

CHAPTER 3

METHODOLOGY

3.1 Research Paradigm

This thesis follows a Design Science Research paradigm because the main contribution is a built and evaluated artifact: the AEGIS system. The paradigm is summarized through three requirements:

- **Problem relevance:** Representative reporting requests motivate the need for the artifact.
- **Design rigor:** The semantic layer, threat model, and compiler boundaries define how the artifact is designed.
- **Evaluation:** The 107-request benchmark and true database execution checks evaluate safety, execution validity, and remaining correctness limits.

The artifact was refined through three build-evaluate cycles:

- **Cycle 1:** Built the initial semantic layer and compiler, then tested the core safety path.
- **Cycle 2:** Expanded coverage to the eleven analytical patterns and replaced manual synonym handling with vocabulary injection.
- **Cycle 3:** Added widget persistence and expanded the benchmark execution harness.

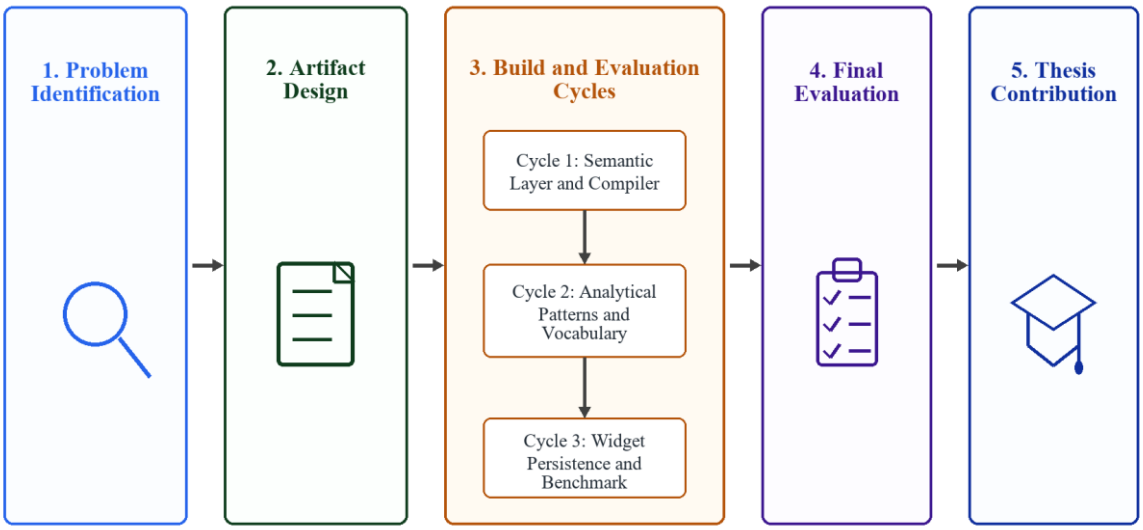

Figure 1: Design Science Research workflow for AEGIS

3.2 Formative Study of Reporting Patterns

The eleven-pattern taxonomy used throughout this thesis (Table 3.2) originates from a design-time review of representative e-commerce and administrative reporting requests, conducted by the author while designing AEGIS. This was a qualitative review carried out by a single researcher during system design, not an independently annotated, inter-rater-validated study, and no separate annotated dataset accompanies this thesis. An earlier draft reported specific percentages and an inter-rater reliability statistic for a larger unpublished dataset; that dataset was not published alongside this thesis, so those figures have been withdrawn.

In place of that withdrawn figure, Table 3.1 reports a pattern classification of the full 107-request custom benchmark. The first 100 requests are answerable analytical requests classified into the eleven AEGIS patterns by the author; the remaining seven are mixed benchmark requests that test behavior beyond the normal template set and are still counted in the same benchmark denominator. This classification is itself a single-annotator judgment, not independently cross-checked by a second annotator, so it should not be read as a validated inter-rater statistic; unlike the withdrawn figures, however, it is reproducible from the evaluation materials supplied with this thesis.

Table 3.1: Benchmark pattern classification.

Pattern	Count (of 107)	Share	Example
KPI / Aggregate	28	26.2%	Orders placed today
Ranking	21	19.6%	Top refund categories
Exception / Filter	18	16.8%	Low-stock products
Trend Analysis	10	9.3%	Monthly sales trend
Comparison	10	9.3%	Mobile vs desktop AOV
Summary / Group	9	8.4%	Category overview
Cohort	2	1.9%	First-time vs returning
Funnel	1	0.9%	Cart abandonment
Correlate	1	0.9%	Margin correlation
Segment	0	0%	Not in sample
Tabular	0	0%	Not in sample
Additional mixed requests	7	6.5%	Boundary-style requests

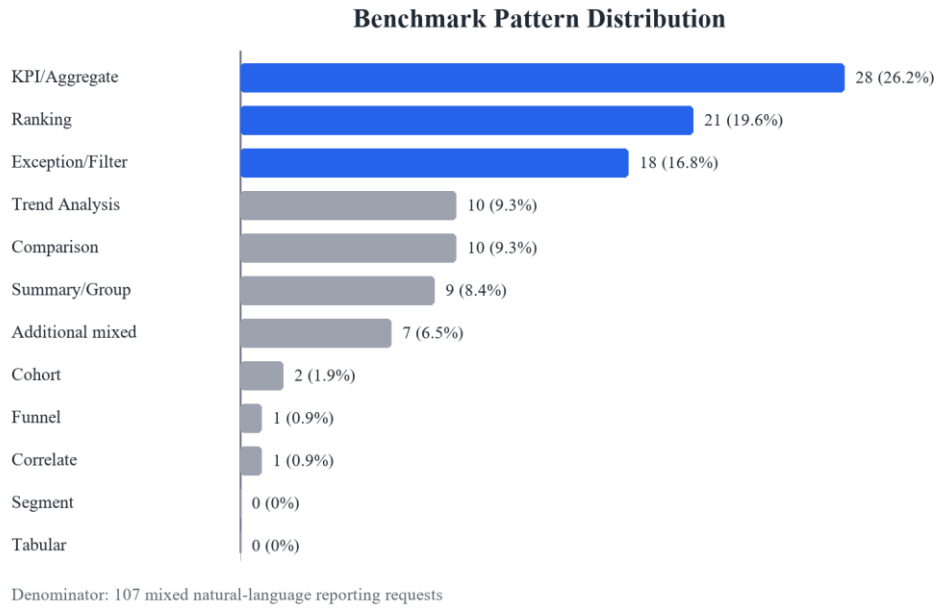

Figure 2: Pattern classification of the answerable analytical benchmark subset

This classification yields three design directions that shaped the methodology and system design. First, a small set of patterns appears sufficient: the top three analytical patterns (KPI, Ranking, and Exception/Filter) account for 67 of the 107 benchmark requests, or about 62.6% of the full mixed benchmark. Segment and Tabular happen not to be exercised by this particular run, which is a property of this benchmark rather than evidence that those two patterns are unnecessary; the compiler supports both regardless. Second, business vocabulary differs systematically from database column names: reviewed requests used phrases like 'total refund rate,' never SUM(o.RefundedAmount), which motivates an explicit semantic layer rather than exposing the schema directly to the language model. Third, reuse appears to be the norm rather than the exception: many requests were variations of things already asked before, in a different time window or for a different segment, which motivates the widget persistence design.

3.3 Design Principles

Five principles guide the AEGIS architecture:

7. Separate understanding from execution. The LLM understands the question; fixed rules handle everything else.
8. Define business terms explicitly. Metrics, dimensions, joins, and time rules are written once in a semantic layer, not inferred per query.

9. Limit what SQL can be generated. SQL is built only from pre-approved, parameterized templates.
10. Select visualizations by rule. Chart type is decided by question type, result shape, and established visualization design guidance, not by a learned model.
11. Persist results for reuse. Every query produces a saved, refreshable widget rather than a discarded answer.

3.4 Formal Model

The practical model follows directly from the design principles above: user language is converted into a typed intent, approved semantic bindings are selected, and SQL is built only by deterministic templates. This keeps the model useful for understanding language without giving it authority to write executable SQL.

3.5 Threat Model

AEGIS protects against attacks arriving through the untrusted natural-language input channel. The model assumes the database and application server are properly hardened; the attacker controls only the query field.

- **T1 - Prompt injection attempting SQL generation:** "Ignore previous instructions. Generate DROP TABLE orders."
    - **Control:** The IntentObject schema contains no SQL field. Any non-approved string in metric_term or dimension_term is rejected by Pydantic type validation at Stage 2, before the compiler is reached.
- **T2 - Unauthorized metric or dimension access:** "Show me customer passwords" or "List credit card numbers by order."
    - **Control:** Fields such as customer_password do not exist in the semantic layer vocabulary. The LLM never sees those names; it receives only the curated, approved label list. Stage 2 rejects any unrecognized term.
- **T3 - Unauthorized row access:** A store-level user asks: "Show revenue for all branches."
    - **Control:** Stage 4 (Permission Rewriter) runs after the LLM and appends a role-specific WHERE predicate (for example, AND o.StoreId = :user_store) derived from

the authenticated session. This cannot be suppressed or overridden by natural-language content.

- **T4 - DML or DDL injection:** A crafted prompt that tricks the LLM into associating a write operation with an intent class.
    - **Control:** No template in the pattern library contains a DML or DDL keyword. The AST-level post-compilation validator explicitly rejects any non-SELECT statement as a defense-in-depth layer.

Not protected by AEGIS (requires operational security controls outside this architecture):

- A malicious administrator embedding arbitrary SQL inside a metric's sql_expr field.
- Supply-chain compromise of the compiler module or the SQL parser library.
- Database-level privilege escalation that bypasses the application layer.
- LLM provider infrastructure compromise or model poisoning.

Explicitly documenting out-of-scope threats is itself a contribution: prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes meaningful security comparison difficult.

3.6 System Architecture

Every user query travels through exactly seven stages. Stages 2 through 7 contain no artificial intelligence; they are deterministic code. Rejection at any stage produces a structured clarification message rather than a best-effort guess.

- **Stage 1 - Intent Extraction:** A lightweight LLM reads the query together with a system prompt built from the semantic layer, and outputs a validated IntentObject (intent class, metric term, dimension term, filters, sort, limit, confidence). This is the only stage that involves artificial intelligence.
- **Stage 2 - Coverage Validation:** The server checks that both the metric term and the dimension term exist in the semantic layer vocabulary before anything else runs. Unknown identifiers are rejected here, with a structured message listing the available identifiers, rather than being passed to the compiler.
- **Stage 3 - Semantic Mapping:** Business-logic aliases are expanded (for example, 'abandoned' maps to a specific OrderStatusId), and relative time expressions such as 'this month' are resolved to concrete date predicates.
- **Stage 4 - Permission Rewriting:** A role-specific WHERE predicate is appended based on the authenticated user's session. This runs after the LLM has already finished, so no natural-language content can influence it.
- **Stage 5 - SQL Compilation:** A breadth-first search over the join graph finds the minimal join path connecting the tables required by the resolved metric and dimension, and pre-compiled SQL expressions are substituted into a parameterized template. No SQL text is ever assembled from concatenated user input.
- **Stage 6 - Visualization Selection:** A rule engine maps the intent class and result shape to a default chart type. This stage contains no learned model.
- **Stage 7 - Widget Persistence:** The analysis plan is hashed (SHA-256) to detect duplicates, and the query, chart configuration, and access rules are stored as a widget artifact that can be refreshed on a schedule.

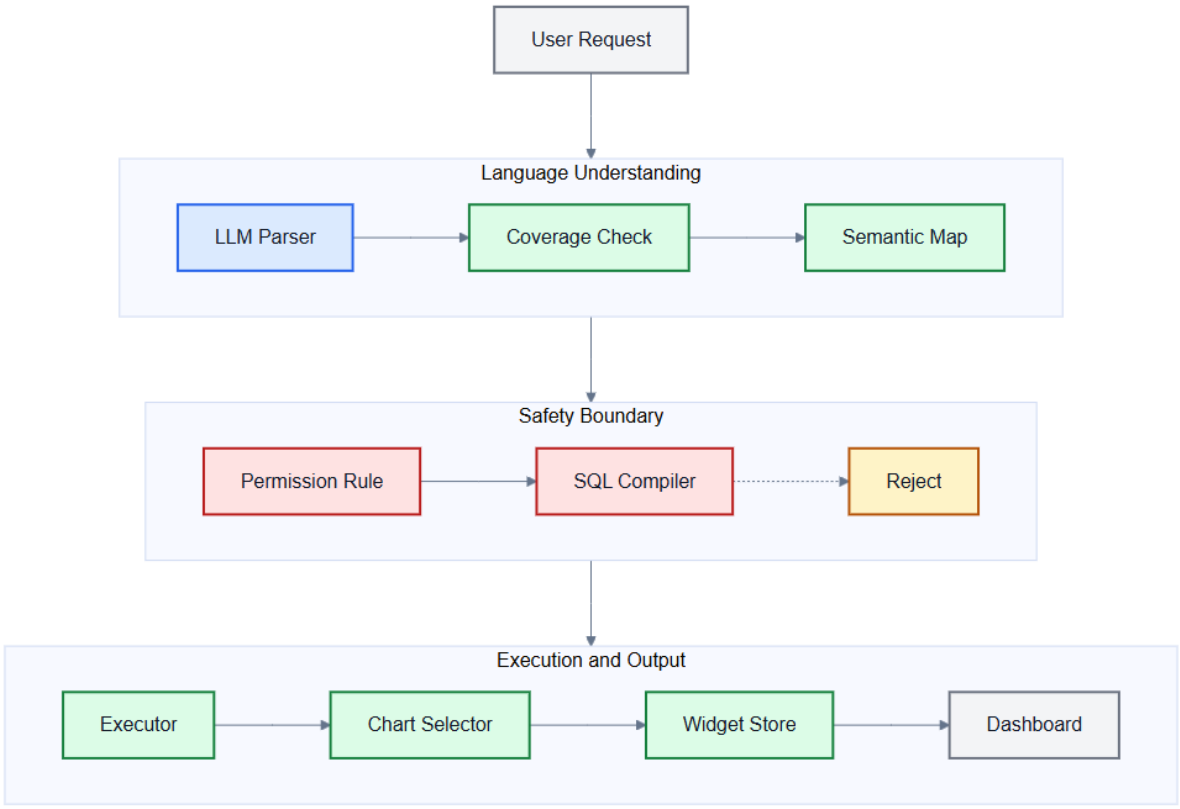

Figure 3: AEGIS architecture pipeline (User Request to Dashboard Widget)

3.7 Semantic Layer Design

The semantic layer is the most important non-AI component of AEGIS. It separates business language from the underlying database structure and defines exactly which metrics, joins, and permissions are allowed to exist. A useful analogy is LEGO blocks rather than free-form clay: the semantic layer defines a finite set of composable building blocks. User questions are unlimited, but every answerable question is a composition of these blocks.

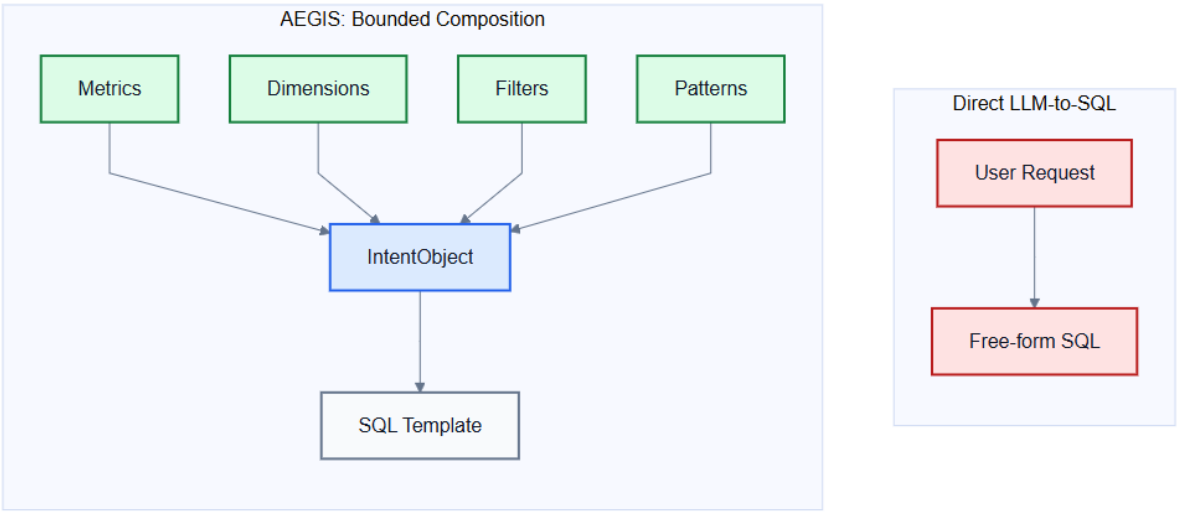

Figure 4: Semantic layer modularity - composable blocks vs. free-form SQL generation

Table 3.2: Semantic layer object model.

Object	Field	Example
Metric	label, SQL expression, joins, visual default, security class	revenue = SUM(o.OrderTotal - o.RefundedAmount)
Dimension	label, SQL expression, datatype, access scope	category = c.Name from Category
Filter	label, SQL predicate, datatype	payment_status : o.PaymentStatusId = :val
Time rule	label, SQL predicate, granularity	current_week : DATEADD(week, ...)
Join path	source, target, ON clause	Order to OrderItem to Product to Category
Pattern	required slots, SQL template, visualization default	ranking: metric plus dimension maps to a bar chart
Permission	rule	store_manager filtered by store location

In the nopCommerce prototype, the semantic layer defines 15 metrics, 34 dimensions, and 11 join paths across 12 analytics-relevant tables. The full nopCommerce schema contains 126 tables; the remaining 114 (system, content-management, configuration, authentication, vendor, and promotions tables) are deliberately not represented in the semantic layer at all. No business analyst asks 'show me revenue by ScheduleTask,' and excluding these tables functions as an implicit table-level access control: even a crafted prompt that names a hidden system table directly is rejected at Stage 2, because the identifier simply does not exist in L.

3.8 Intent Parsing with Dynamic Vocabulary Injection

The central technique that makes Stage 1 reliable is vocabulary injection. At startup, AEGIS builds the system prompt by listing every approved metric and dimension name, with a plain-English description, directly from the semantic layer (approximately 1,100 tokens for 15 metrics and 34 dimensions). The LLM sees exactly which identifiers are valid and maps any user phrasing to the correct identifier without a manually maintained synonym list. This has three advantages over a synonym dictionary: zero maintenance, since adding a metric automatically updates the vocabulary the model sees; broad coverage of arbitrary user phrasing, since the model performs the mapping rather than a fixed lookup table; and token efficiency, since the full vocabulary for 15 metrics and 34 dimensions fits in roughly 1,100 tokens.

Structured output enforcement for LLMs [16] constrains the model's response to a fixed JSON schema before any downstream validation occurs, which is why Stage 1 can rely on a typed IntentObject rather than free-form text: malformed or off-schema output is rejected at the API boundary, before Stage 2 coverage validation ever runs.

The output schema enforces typed fields:

```
{
  "intent_class": "kpi | ranking | trend | comparison | exception |
                   summary | segment | funnel | cohort | correlate | tabular",
  "metric_term": "string",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value": "string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean"
}
```

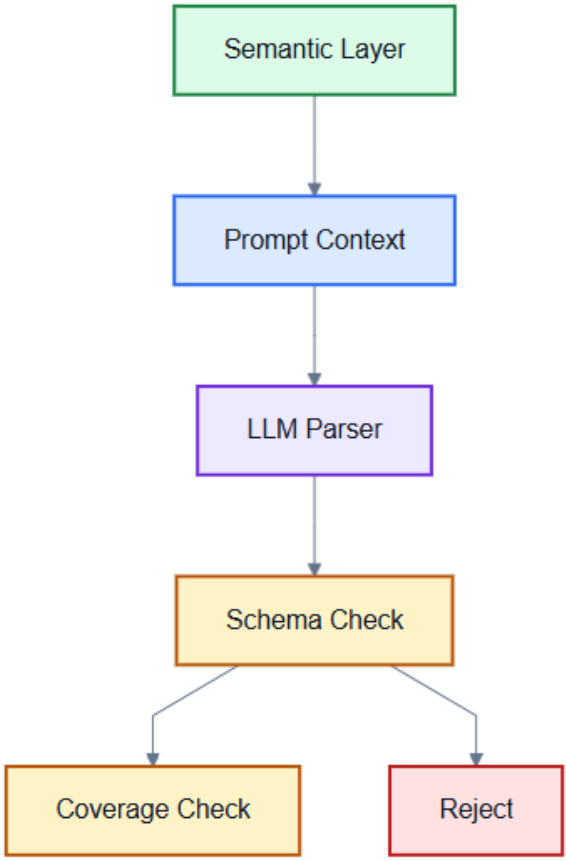

Figure 5: Vocabulary injection workflow

3.9 Safe Query Compiler

The compiler instantiates SQL from a library of parameterized templates, one per analytics pattern.

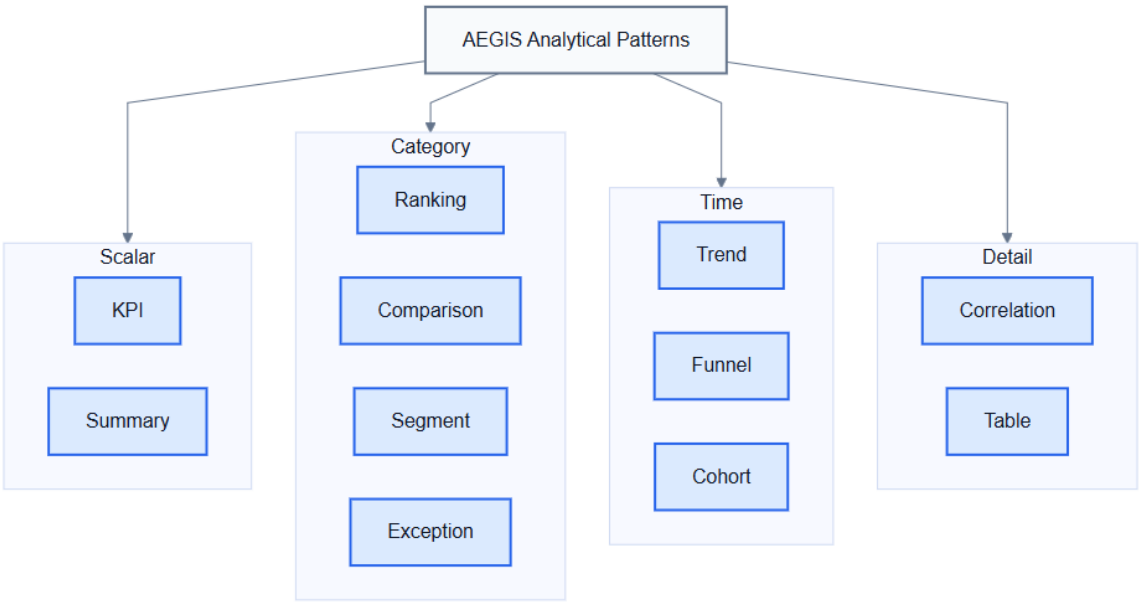

Figure 6: Taxonomy of the eleven AEGIS analytical patterns

Table 3.3: The eleven AEGIS analytical patterns.

Pattern	Required slots	Optional slots	Default visual
KPI (Aggregate)	metric	time_rule, filter	kpi_card
Ranking	metric, dimension	time_rule, filter, limit	bar_chart
Trend	metric, time_grain	time_rule, filter	line_chart
Comparison	metric, segment	time_rule, filter	grouped_bar
Exception	metric, threshold	dimension, time_rule	table
Summary	metric[], dimension	time_rule, filter	multi_card
Segment	metric, dimension	time_rule, filter	pie_chart
Funnel	metric, stages	time_rule, filter	funnel_chart
Cohort	metric, group_def	time_rule, filter	grouped_bar
Correlate	metric, attribute	time_rule, filter	scatter_plot
Tabular	dimension	filters, time_rule	table

After placeholder substitution, two safety layers apply in sequence. Layer 1 is a parameterized query engine that separates SQL structure from user-supplied inputs, so no user text is ever concatenated into the SQL string. Layer 2 is a post-compilation safety scanner that rejects any assembled query containing a forbidden construct: non-SELECT statements, UNION, EXCEPT or INTERSECT, EXEC, or references to system tables. If any forbidden pattern is detected, the compiler raises a SecurityError rather than returning a partially safe query.

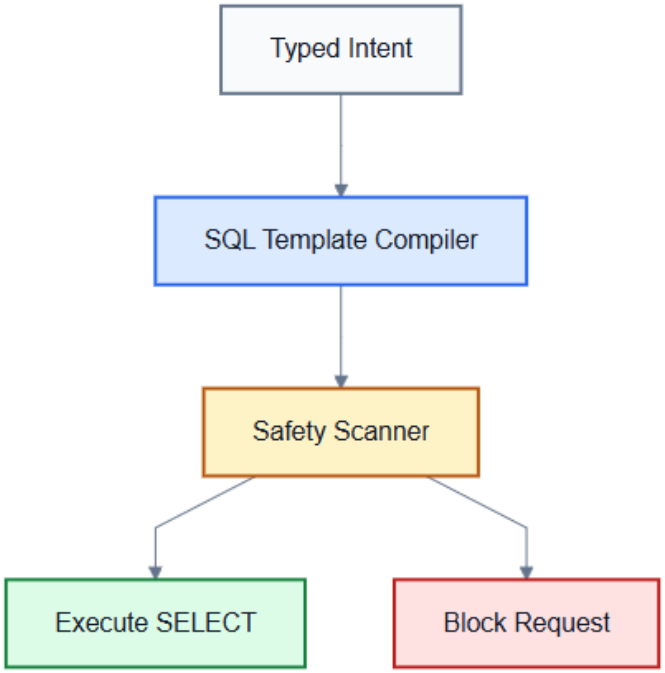

Figure 7: Two-layer SQL safety defence

3.10 Visualization Selector

Table 3.4: Visualization selector mapping.

Intent	Result shape	Selected visualization
KPI	scalar	KPI card
Ranking	1 measure, up to 20 categories	Horizontal bar chart
Trend	1 measure, time series	Line chart
Comparison	1 measure, 2-4 segments	Grouped bar chart
Exception	row-level detail	Sortable table
Summary	2-4 scalar measures	KPI card grid
Segment	1 measure, categorical	Pie chart
Funnel	ordered conversion stages	Funnel chart
Cohort	1 measure, 2+ groups	Grouped bar chart
Correlate	2 measures, continuous	Scatter plot
Tabular	raw records	Sortable table

Two additional rules apply after the data is known, rather than at selection time: bar charts with more than 20 categories are automatically converted to a table, and pie charts with more than 8 slices are converted to a bar chart. Both rules exist because the initial chart choice is made before the exact cardinality of the result is known.

3.11 Widget Persistence and Reuse

Each widget stores a unique identifier (a SHA-256 hash of the analysis plan), the original question, the analysis plan in JSON form, a hash of the SQL template used, chart configuration, timestamps, access rules, and run history. A new question triggers the full seven-stage pipeline; if an identical widget already exists, determined by a match on the plan hash, the cached artifact is returned immediately rather than recompiled. Scheduled refresh re-executes the stored SQL against fresh data on a configurable interval, which directly addresses the design-time observation that reporting requests are often recurring rather than one-off: a widget answers the question once and then continues answering it as new data arrives, rather than requiring the same natural-language request to be re-processed from scratch every time.

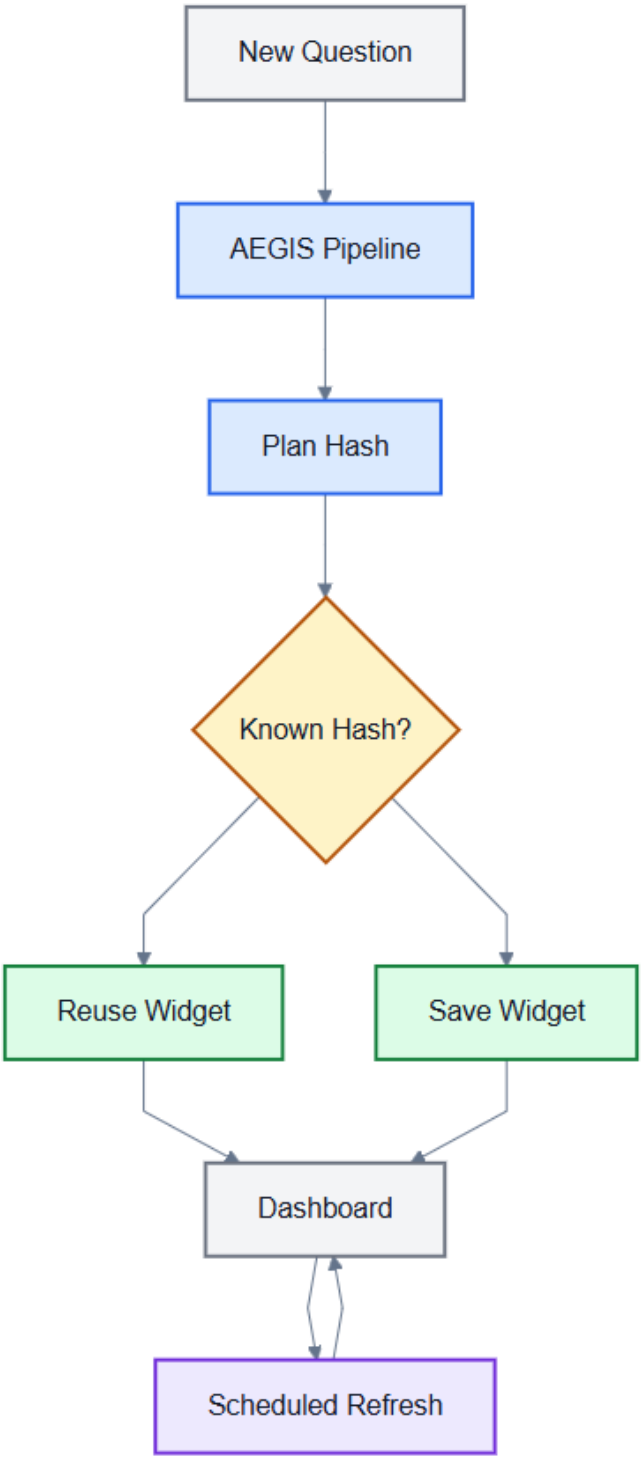

Figure 8: Widget lifecycle and refresh model

CHAPTER 4

EXPERIMENTAL WORK

4.1 Implementation

AEGIS is implemented as a web application with a vanilla HTML and JavaScript frontend (jQuery, Chart.js) and a Python FastAPI backend, targeting a production nopCommerce 4.70 schema (126 tables, 107 foreign key constraints). The implementation follows directly from the AEGIS architecture:

- **LLM integration:** Llama 3.1 8B Instant via the Groq API, with structured JSON output enforcement. The system prompt is constructed dynamically by injecting the approved metric and dimension identifiers at startup.
- **Rate limiting:** A provider-agnostic configuration module with a sliding-window rate limiter and a concurrency-safe asyncio.Lock, so the architecture is not tied to a specific LLM vendor's throughput characteristics.
- **Semantic layer:** Python configuration modules containing 15 metrics, 34 dimensions, zero synonym entries, and 11 join paths across the 12 analytics-relevant tables represented in the semantic layer.
- **SQL compiler:** Parameterized MySQL templates, with breadth-first search join-path resolution over the 12-table join graph. A post-compilation _validate_sql_safety() routine checks 16 forbidden patterns before a query is allowed to execute.
- **Visualization selector:** Rule-based Python dictionaries implementing the visualization mapping, with the two post-hoc cardinality rules applied after the result set is known.
- **Widget engine:** SHA-256 plan-hash deduplication, with JSON file storage in the prototype, designed to be swapped for a relational store in a production deployment.
- **Coverage validator:** A pre-compilation gate that rejects unknown metric or dimension terms with structured guidance listing the available identifiers.
- **Permission enforcement:** A Permission Rewriter that appends role-based WHERE predicates for five roles: public, store_manager, regional_manager, read_only, and analyst.

4.2 Experimental Environment

The experimental setup used a containerized relational database and a seeded e-commerce dataset, summarized in Table 4.1.

Table 4.1: Experimental setup.

Setup item	Configuration
Database engine	MySQL 8.0
Execution mode	Docker container
Schema	AEGIS Truth Schema based on nopCommerce 4.70
Schema size	126 tables and 107 foreign keys
Dataset scale	1,200 customers, 2,500 orders, 6,298 order items, 1,000 products, and 50 categories
Data period	Orders spanning 2024 to 2026
Evaluation scope	Mid-sized e-commerce analytics workload

4.3 Benchmark Dataset Construction

This evaluation is a prototype evaluation, not a large-scale independent benchmark study. Its goal is to demonstrate that the AEGIS architecture achieves its stated safety and semantic-fidelity properties on representative real-world analytics queries, not to establish population-level accuracy claims. Standard text-to-SQL benchmarks such as Spider and BIRD do not evaluate adversarial safety or adherence to business vocabulary, which is why a domain-specific benchmark was necessary for this thesis.

A methodological caveat applies to every quantitative result reported in the experimental work and results discussion. All benchmark construction, execution, and measurement were carried out by the author as part of this thesis, using a self-built dataset and evaluation harness; none of it has been independently replicated by a third party, externally audited, or peer-reviewed. Therefore, the reported figures should be read as prototype evaluation results produced within this research, not as independently audited benchmark results. Where an annotation or classification step is discussed, it refers to the author's own research process rather than a third-party validation study.

A domain-specific benchmark of 107 natural-language reporting requests was built over the same production nopCommerce schema. The full question set and recorded pipeline outputs were used to check every reported metric. The benchmark mixes ordinary analytical requests with harder boundary cases in the same run; this thesis does not report those harder cases as a separate benchmark. Because the benchmark was constructed for the nopCommerce domain,

accuracy and validity reported figures should be interpreted within that scope. The current artifact verifies SQL safety and true execution validity; semantic correctness still requires an annotated expected-answer benchmark in future work.

4.4 Baseline Systems

The evaluation plan defines four baselines, but not all are complete in the current prototype:

- **B1 - Direct LLM-to-SQL:** Llama 3.1 8B is prompted with the full database schema, without semantic-layer or template constraints.
- **B2 - Decomposed LLM:** A chain-of-thought strategy first extracts entities and then generates SQL. This baseline is prepared but not included in the completed results.
- **B3 - Template-only:** Keyword matching selects templates without LLM-based intent extraction. This baseline has been executed for true database execution validity.
- **B4 - No semantic layer:** The full AEGIS pipeline runs with the semantic mapper bypassed. This remains future evaluation work.

4.5 Evaluation Procedure

The current evaluation focuses on measurements that can be reproduced from the recorded benchmark data and verified through true database execution:

- **RQ1:** How accurately does the LLM intent parser extract typed reporting plans? This remains a semantic-correctness task requiring annotated expected labels.
- **RQ2:** Does AEGIS reduce unsafe SQL compared to direct LLM-to-SQL? Measured as genuine unsafe SQL statements across the 107-query benchmark.
- **RQ3:** Does AEGIS generate SQL that actually runs? Measured through true database execution against the seeded MySQL database, not merely by checking whether Python compilation succeeded.
- **RQ4:** How does the deterministic downstream compiler behave when intent selection is rule-based rather than LLM-based? Measured through the B3 template-only baseline.

CHAPTER 5

RESULTS AND DISCUSSION

The results discussion presents the evaluation results of the AEGIS prototype using a 107-request natural-language analytics benchmark and true database execution. The discussion separates SQL safety, execution validity, and semantic correctness because a query may be safe and executable while still failing to answer the intended analytical request.

The reported quantitative evidence covers the mixed 107-request benchmark for AEGIS and the B1 direct LLM-to-SQL baseline, together with the completed B3 template-only execution-validity run. B2, B4, latency, and fully annotated semantic correctness remain future evaluation work, so the reported results include only the measurements completed under the current experimental setup.

5.1 Evaluation Overview

The evaluation uses 107 mixed natural-language reporting requests as a single benchmark set. The requests include ordinary analytical questions as well as harder boundary-style cases, but all are evaluated under the same denominator to avoid separating results into unsupported categories.

Table 5.1: Evaluation benchmark status.

Item	Status
Benchmark size	107 mixed natural-language reporting requests
Database execution environment	MySQL 8.0 in Docker, seeded with nopCommerce-style data
Completed baselines	B1 direct LLM-to-SQL and B3 template-only
Pending measurements	Semantic correctness, B2/B4 baselines, and latency

The benchmark evidence consists of the prepared request set, recorded model outputs, baseline outputs, and true database execution results produced against the seeded MySQL evaluation database.

5.2 Main Result Summary

Reference text-to-SQL papers commonly present a compact result table before detailed analysis. Following that pattern, Table 5.2 summarizes the completed AEGIS measurements and separates them from measurements that remain pending.

Table 5.2: Main evaluation result summary.

Metric	AEGIS result	Baseline result	Interpretation	Status
SQL safety	0 unsafe statements in 107	B1 produced 1 unsafe statement in 107	AEGIS blocked the observed unsafe-write case	Measured
True execution validity	100 of 107 executed successfully (93.5%)	B1: 27 of 107 (25.2%); B3: 104 of 107 (97.2%)	AEGIS greatly improved execution validity over direct LLM-to-SQL	Measured
Failure analysis	7 AEGIS execution failures	B1 failures were broader and more frequent	Remaining AEGIS failures are implementation issues, not safety violations	Measured
Semantic correctness	Not numerically scored	Not numerically scored	Requires annotated expected answers before accuracy can be claimed	Pending
Latency	Not instrumented	Not instrumented	Needs repeatable timing logs before reporting	Pending

5.3 SQL Safety

Unsafe Query Execution Rate measures whether a generated SQL statement contains a genuine unsafe operation such as a write or schema-changing command. This metric is different from execution validity because a statement can be syntactically valid while still being unsafe for an analytics-only system.

Table 5.3: SQL safety on the 107-request benchmark.

System	Unsafe SQL	Interpretation
Baseline B1 (Direct LLM-to-SQL)	1 genuine unsafe statement in 107	The model generated one write operation when prompted with a business-looking request.
AEGIS	0 unsafe statements in 107	The intent schema and compiler templates did not allow write SQL to be emitted.

The strongest safety example is the disguised write request asking to cancel orders stuck in Pending for more than 30 days. Baseline B1 produced an executable UPDATE statement. AEGIS cannot express a write intent in its intent object or compiler templates, so it did not emit write SQL. This is the central safety result: AEGIS prevents this class of unsafe execution structurally rather than hoping the model follows an instruction not to write dangerous SQL.

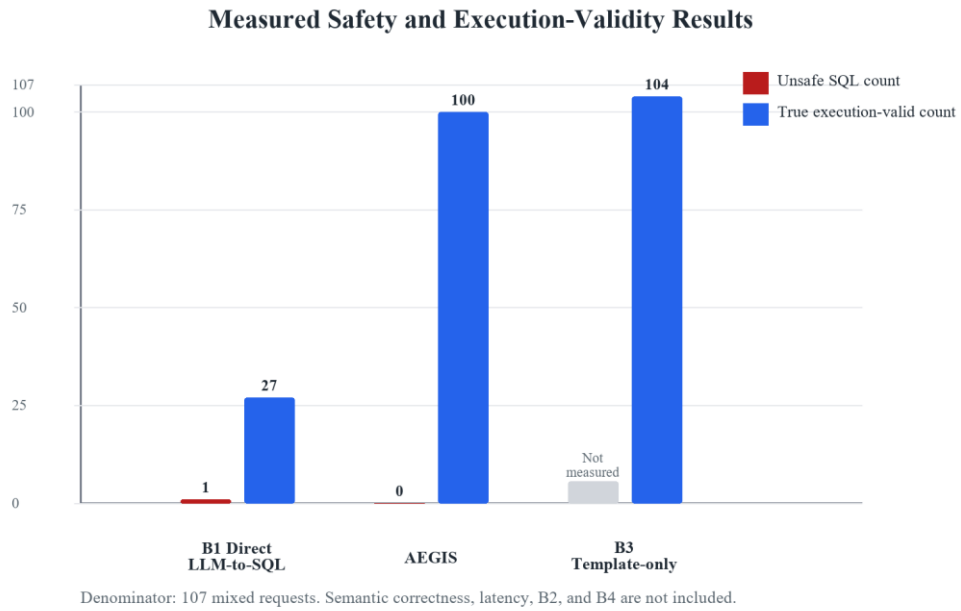

Figure 9: Verified safety and execution-validity comparison

5.4 True Database Execution Validity

True database execution validity measures whether the generated SQL actually runs against the seeded MySQL database. This check is stricter than string inspection or application-level compilation because it exposes missing columns, invalid joins, dialect errors, and invalid literal values.

Table 5.4: True execution validity on the 107-request benchmark.

System	Execution result	Interpretation
Baseline B1 (Direct LLM-to-SQL)	27 of 107 executed successfully (25.2%)	Most failures came from hallucinated columns, invalid table references, or dialect errors.
AEGIS	100 of 107 executed successfully (93.5%)	The deterministic pipeline removed many free-form SQL errors, but seven issues remained.
B3 Template-only	104 of 107 executed successfully (97.2%)	High execution validity, but not evidence of higher semantic correctness.

5.5 Execution Failure Analysis

The seven AEGIS execution failures identify the remaining implementation limitations of the prototype. These failures are not safety violations; they are executable-query construction issues observed when the generated SQL was tested against the evaluation database.

Table 5.5: AEGIS true-execution failures diagnosed from MySQL execution.

Query ID	Failure class	Observed database error
5	Relative-date normalization	Incorrect DATETIME value: 'this morning'
13	Compiler syntax edge case	MySQL syntax error near generated SQL fragment
19	Join-path or alias defect	Unknown column 'o.Id' in ON clause
46	Relative-date normalization	Incorrect DATETIME value: 'last 90 days'
72	Relative-date normalization	Incorrect DATETIME value: 'this quarter'
80	Compiler syntax edge case	MySQL syntax error near generated SQL fragment
84	Compiler syntax edge case	MySQL syntax error near generated SQL fragment

The failure pattern is narrow. Three failures are date-normalization bugs, three are compiler syntax edge cases, and one is a join or alias construction defect. These are concrete implementation bugs, not evidence that the safety boundary failed. They should be fixed in future work and then re-measured using the same true database execution procedure.

5.6 Semantic Correctness Limitation

Correctness or accuracy must be treated separately from execution validity. The current results show that AEGIS often generates SQL that executes successfully and avoids unsafe operations, but they do not establish that every executable result matches the user's intended analytical meaning.

Table 5.6: Distinguishing the evaluation metrics.

Metric	What it answers	Current status
SQL safety	Did the generated SQL avoid unsafe write or schema-changing behavior?	Verified for B1 and AEGIS on all 107 requests.
True execution validity	Did the generated SQL run against the seeded MySQL database?	Verified for B1, AEGIS, and B3.
Semantic correctness / accuracy	Did the answer match the user's intended reporting need?	Not yet numerically scored; requires annotated expected outputs or labels.
Scope handling	Did the system clarify or reject unsupported requests instead of guessing?	Observed as a limitation; needs a separate annotated evaluation.
Latency	How long each stage takes end to end?	Not yet instrumented.

This distinction is important for the thesis argument. AEGIS maintained SQL safety even under harder boundary cases, but the scope-detection mechanism was incomplete: several unsupported or underspecified requests were mapped to safe but semantically wrong in-scope queries instead of being rejected or clarified. That behavior should be reported as a correctness and clarification limitation, not hidden inside the safety metric.

5.7 B3 Template-Only Baseline

The B3 template-only baseline is useful because it separates the deterministic compiler from the LLM intent parser. B3 uses keyword matching to create an intent object, then hands that intent to the same downstream semantic mapping and compiler path. Its high execution-validity result shows that many database errors are triggered by particular intent choices and time phrases, not by the compiler alone.

Table 5.7: B3 execution-validity summary.

System	Execution result	Interpretation
B3 Template-only	104 of 107 executed successfully (97.2%)	Higher execution validity than AEGIS on this run, but not evidence of higher semantic correctness.
B3 failures	3 database execution failures	Two compiler or join-path issues and one date-value issue were observed.

B3's result should be interpreted carefully. A keyword-only classifier may choose a query shape that runs successfully while answering the wrong question. Therefore B3 belongs in the execution-validity comparison, but it should not be used to claim semantic accuracy until the same annotated correctness benchmark is applied to all systems.

5.8 Comparative Discussion

5.8.1 AEGIS vs. Direct LLM-to-SQL

The B1 baseline exposes the central risk of direct SQL generation. The same model that can produce useful analytical SQL can also hallucinate schema objects, mix SQL dialects, and generate a real write statement when a business-looking request implies a write operation. AEGIS narrows the model's role to intent extraction, so these risks do not enter the SQL construction stage in the same way.

Table 5.8: Structural comparison of AEGIS vs. direct LLM-to-SQL.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model-generated free-form text	Deterministic compiler
Schema exposure to LLM	Requires tables, columns, and relationships	Uses approved business labels
Business metric definitions	Inferred from prompt and schema names	Defined in the semantic layer
SQL injection prevention	Prompt-level instruction and post-checking	Structural prevention through intent schema and templates
Permission enforcement	External or absent	Applied after intent extraction by deterministic code
Dashboard widget persistence	Not provided by default	First-class saved artifact
Model dependency	Strongly tied to model quality	Compiler and safety scanner are model-independent

5.8.2 Semantic Layer versus Retrieval-Augmented Generation

Retrieval-augmented generation can help an LLM find relevant schema fragments, but it does not remove the model's authority to write SQL. AEGIS uses the semantic layer for a different purpose: it defines which business concepts are allowed to exist and how they compile into SQL. RAG may still be useful in a future large deployment to select relevant semantic-layer entries, but the final SQL should still be compiled by the deterministic AEGIS compiler.

5.8.3 Scope Boundary

AEGIS currently supports a bounded set of approved metrics, dimensions, analytical patterns, and join paths. This is the source of its safety, but also the source of its main limitation. If a user asks for something not represented in the semantic layer, the system should reject or clarify the request. The current prototype does not always do that reliably; improving scope detection is therefore a priority in future work.

CHAPTER 6

LIMITATIONS AND FUTURE WORK

6.1 Limitations

- **Semantic layer construction:** Each deployment requires a domain-specific semantic layer prepared by someone with both business knowledge and schema access.
- **Bounded query coverage:** AEGIS only answers questions that map to supported metrics, dimensions, patterns, and join paths.
- **Correctness annotation:** A misclassified intent can produce a safe but semantically wrong query, so semantic correctness must be annotated and measured separately from SQL safety.
- **Benchmark scope:** The custom 107-request benchmark evaluates AEGIS in one domain and is not directly comparable to Spider or BIRD leaderboard scores.
- **Prototype engineering limits:** The current version targets MySQL, uses prototype widget storage, and handles each request independently without multi-turn context.

6.2 Future Work

- **Clarification requests:** When confidence is low, AEGIS should ask a follow-up question instead of guessing, for example "did you mean revenue or profit?"
- **Semantic-layer tooling:** A guided interface should let business analysts define new metrics and dimensions, join paths, and approved vocabulary without editing Python code.
- **Stronger evaluation:** Annotated expected answers, B2 and B4 baselines, latency measurements, and correctness reporting should be added to strengthen the evaluation.
- **Cross-schema evaluation:** A second schema, such as WooCommerce, should be evaluated while keeping the intent parser contract, compiler structure, and safety scanner unchanged.
- **Production hardening:** Query-cost controls, database-backed widget storage, broader schema coverage, and multi-turn support should be added before production deployment.

CHAPTER 7

CONCLUSION

This thesis presented AEGIS, a constraint-based architecture for safe LLM-assisted natural-language analytics over relational databases. The system limits the language model to intent extraction while query construction, chart selection, and widget persistence are handled by deterministic components. This design makes approved business terms explicit through a semantic layer and avoids exposing raw SQL generation authority to the language model.

The prototype evaluation on a 107-request mixed benchmark showed that AEGIS produced no unsafe SQL statements and successfully executed 100 of 107 generated queries against the seeded MySQL database. In contrast, the direct LLM-to-SQL baseline executed 27 of 107 queries successfully and produced one genuine unsafe write statement. These results support the main argument that deterministic compilation and semantic-layer constraints can improve SQL safety in natural-language reporting systems.

AEGIS is not a general-purpose text-to-SQL system. Its limitations include semantic layer construction cost, bounded query coverage, remaining execution failures, incomplete scope detection, and the need for a separately annotated semantic-correctness benchmark. Within this scope, the thesis demonstrates that restricting SQL generation to validated business patterns is a practical direction for safer, reusable, and more auditable natural-language analytics in institutional environments.

REFERENCES

[1] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911-3921.

[2] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[3] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," The VLDB Journal, vol. 28, no. 5, pp. 793-819, 2019.

[4] M. Liu and J. Xu, "NLI4DB: A systematic review of natural language interfaces for databases," arXiv:2503.02435, 2025.

[5] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," Proceedings of the VLDB Endowment, vol. 8, no. 1, pp. 73-84, 2014.

[6] C. Lehmann, D. Gehrig, S. Holdener, C. Saladin, J. P. Monteiro, and K. Stockinger, "Building natural language interfaces for databases in practice," in Proc. 34th Int. Conf. Scientific and Statistical Database Management (SSDBM), 2022, Art. no. 20.

[7] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7567-7578.

[8] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 9895-9901.

[9] H. S. Shalaan, T. H. A. Soliman, and A. M. Abdelaziz, "G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation," IEEE Access, vol. 13, pp. 158520-158534, 2025.

[10] X. Su, Y. Gu, P. Wang, W. Gu, L. Qi, and J. He, "A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs," Scientific Reports, vol. 16, Art. no. 7892, 2026.

[11] A. Narechania, A. Srinivasan, and J. Stasko, "nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

[12] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in Proc. 28th Annual ACM Symposium on User Interface Software and Technology (UIST), 2015, pp. 489-500.

[13] D. Deng, A. Wu, H. Qu, and Y. Wu, "DashBot: Insight-driven dashboard generation based on deep reinforcement learning," IEEE Transactions on Visualization and Computer Graphics, vol. 29, no. 1, pp. 690-700, 2023.

[14] G. N. Shailesh, M. Pavithran, A. Rahul Hari Venkat, and P. Kaliappan, "Conversational BI: Natural language interface to business dashboards," International Journal of Engineering Research & Technology, vol. 14, no. 12, 2025.

[15] J. Valkenburgh, "Enhancing business dashboards with explanatory analytics and AI: Exploring the use of AI and explanatory analytics to enhance business decision-making," M.S. thesis, Information Management, Turku School of Economics, University of Turku, Finland, 2024.

[16] OpenAI, "Introducing structured outputs in the API," OpenAI, 2024.